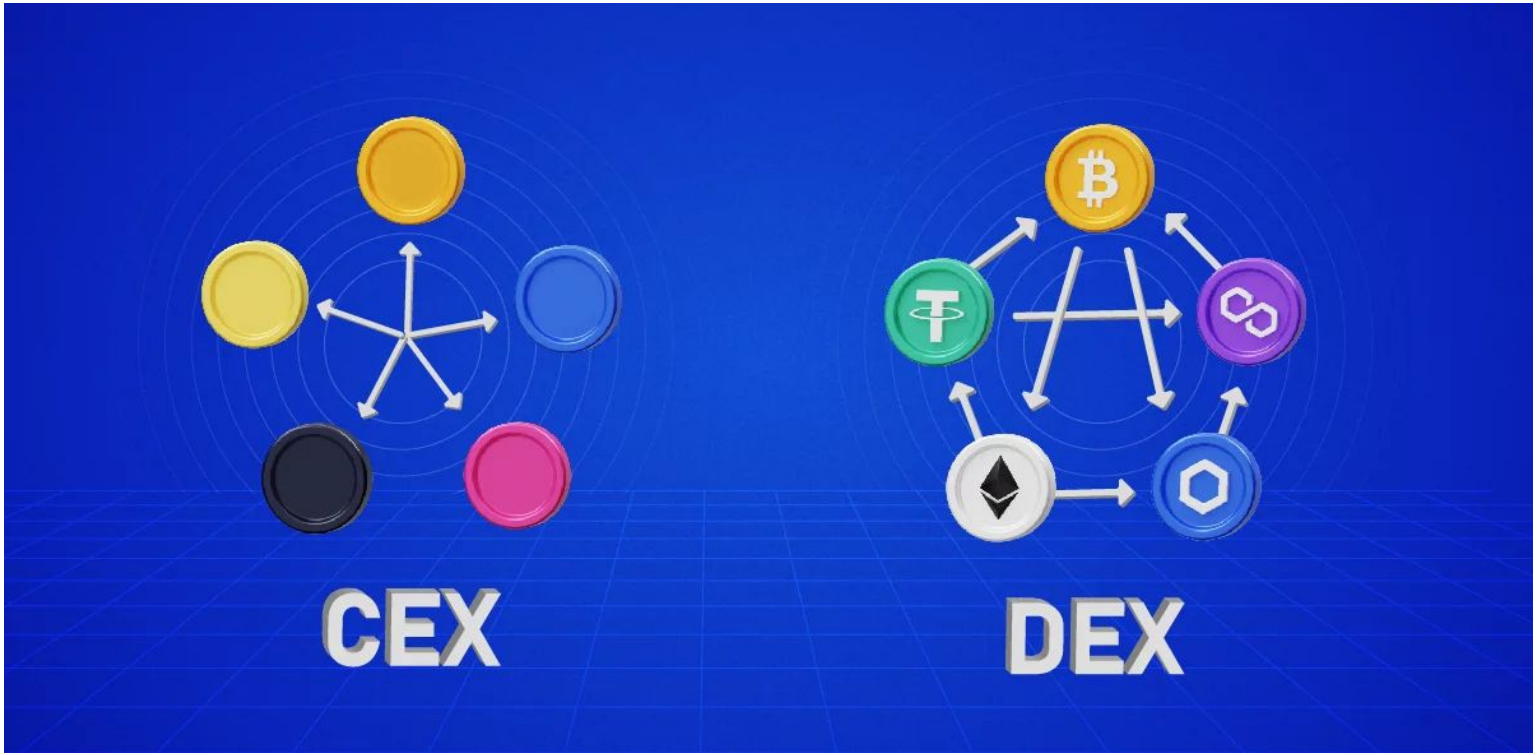
Decentralised Private Limit Order Book (DePLOB)

Research in Blockchain and Cryptocurrency
Chu Wei Rong
A0262363H

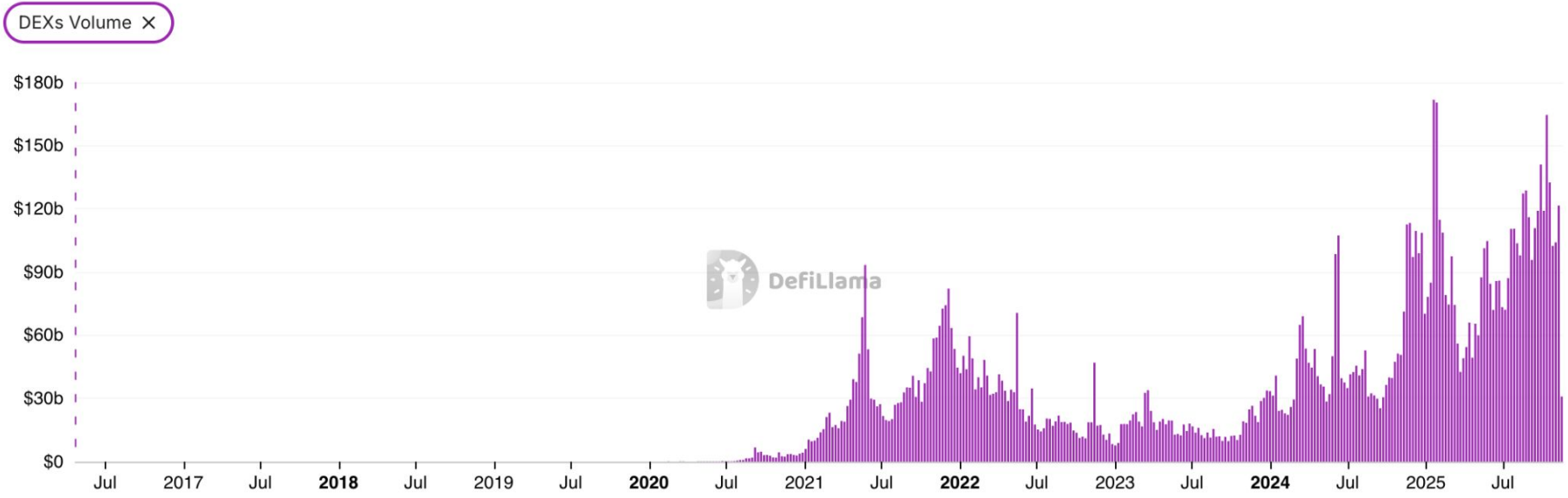
Problem Motivation

Total Market Cap of Cryptocurrencies

Exchanges

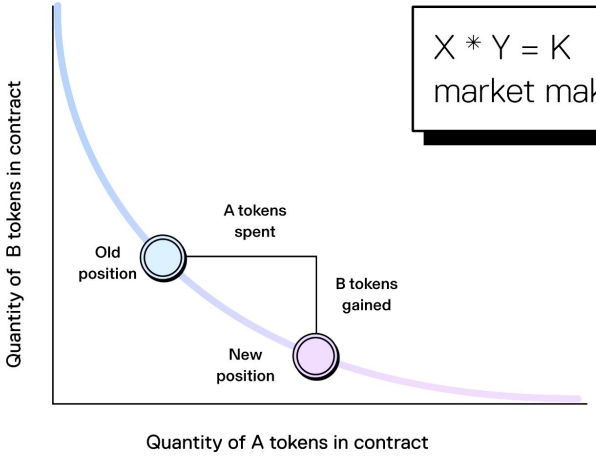


Rise of DEXs


DEXs Volume
$180b
$150b
$120b
$90b
$60b
$30b
$0
DefiLlama
Jul
2017
Jul
2018
Jul
2019
Jul
2020
Jul
2021
Jul
2022
Jul
2023
Jul
2024
Jul
2025
Jul

Dex Types

AMM

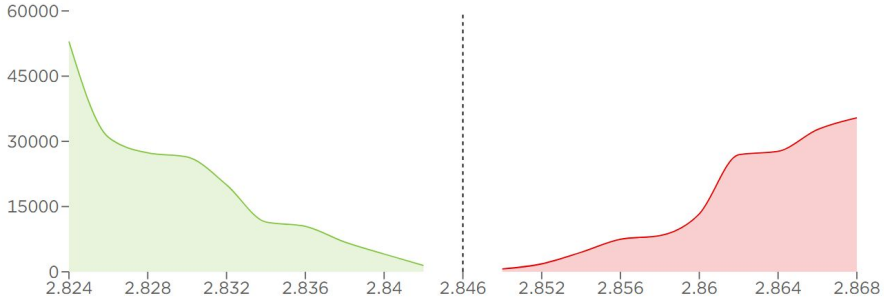


X * Y = K
market maker

Order book

Order	Volume	Buy	Sell	Volume	Order
2	1452	2,842	2,850	647	2
4	2635	2,840	2,852	1179	1
2	2756	2,838	2,854	2656	4
3	3621	2,836	2,856	3009	2
2	979	2,834	2,858	829	3
3	8566	2,832	2,860	5000	1
4	6382	2,830	2,862	13587	3
2	958	2,828	2,864	804	2
2	3596	2,826	2,866	5000	1
5	22030	2,824	2,868	2714	2

29 52975 35425 21



DEXs: Example Trade

TRANSACTION ACTION

Swap 69.93 USDC ($69.91) for 0.183344371 τ wTAO ($68.54) on Uniswap V3

Transaction Hash:	0xfc6f871108a6a54032afefff7a195f59cc01f6945bec3754731cd18442e10fac
Status:	Success
Block:	23776218 1 Block Confirmation
Timestamp:	18 secs ago (Nov-11-2025 01:14:11 PM UTC) | Confirmed within 3 secs
Sponsored:	
From:	0x00840eFfD4319d6993793b88F29a310A92d9A846
Interacted With (To):	0xE592427A0AEce92De3Edee1F18E0157C05861564 (Uniswap V3: Router)
ERC-20 Tokens Transferred: 2	All Transfers Net Transfers
	From Uniswap V3: wTAO-US... To 0x00840eFf...A92d9A846 For 0.183344371 ($68.54) τ Wrapped TAO (wTAO)
	From 0x00840eFf...A92d9A846 To Uniswap V3: wTAO-US... For 69.930407 ($69.92) USDC (USDC)
Value:	0 ETH ($0.00)
Transaction Fee:	0.000040131137918796 ETH ($0.14)
Gas Price:	0.294761127 Gwei (0.000000000294761127 ETH)

DEXs: Example State Change

0xf763Bb34...0c1c17E94 Uniswap V3: wTAO-U...

Storage 2

Storage Address: 0x0000000000000000000000000000000000000000000000000000000000000000

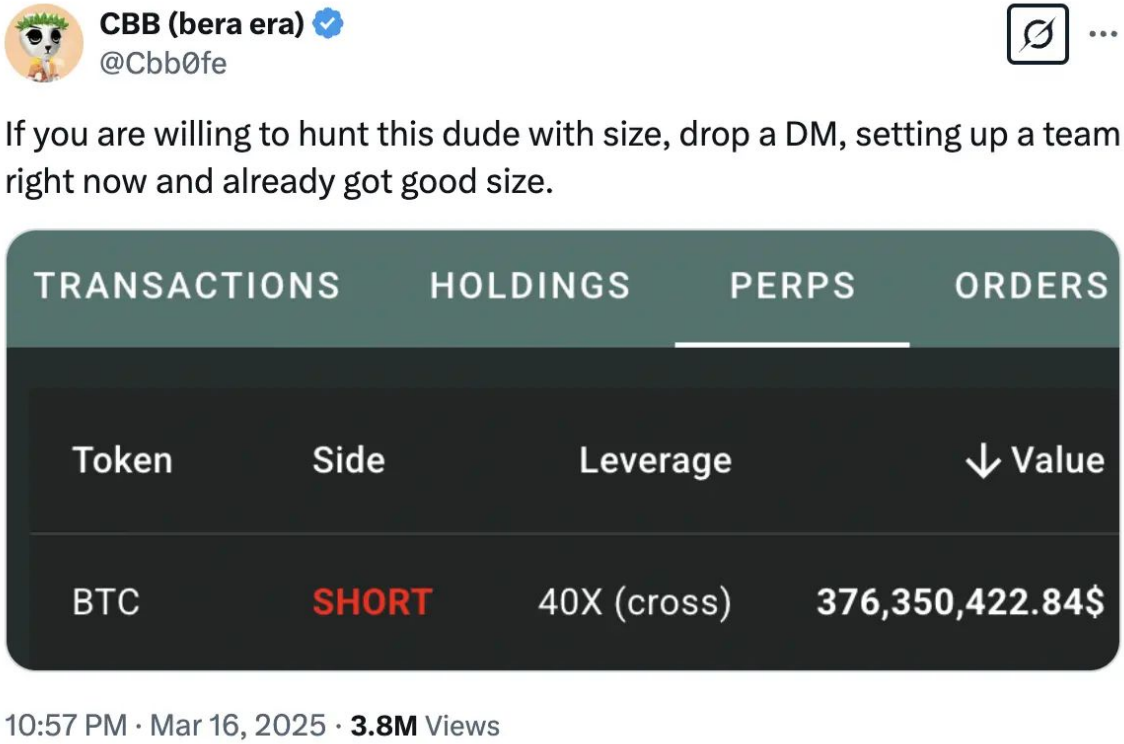
Offset	Variable	Data Type	Before	After
N/A	N/A	N/A	0x000100000500050004ffd9f400000000000000009d4f30e0c8a6eb27051737b9	0x000100000500050004ffd9f400000000000000009d4f9d8a58215df9715a0302

Storage Address: 0x0000000000000000000000000000000000000000000000000000000000000002

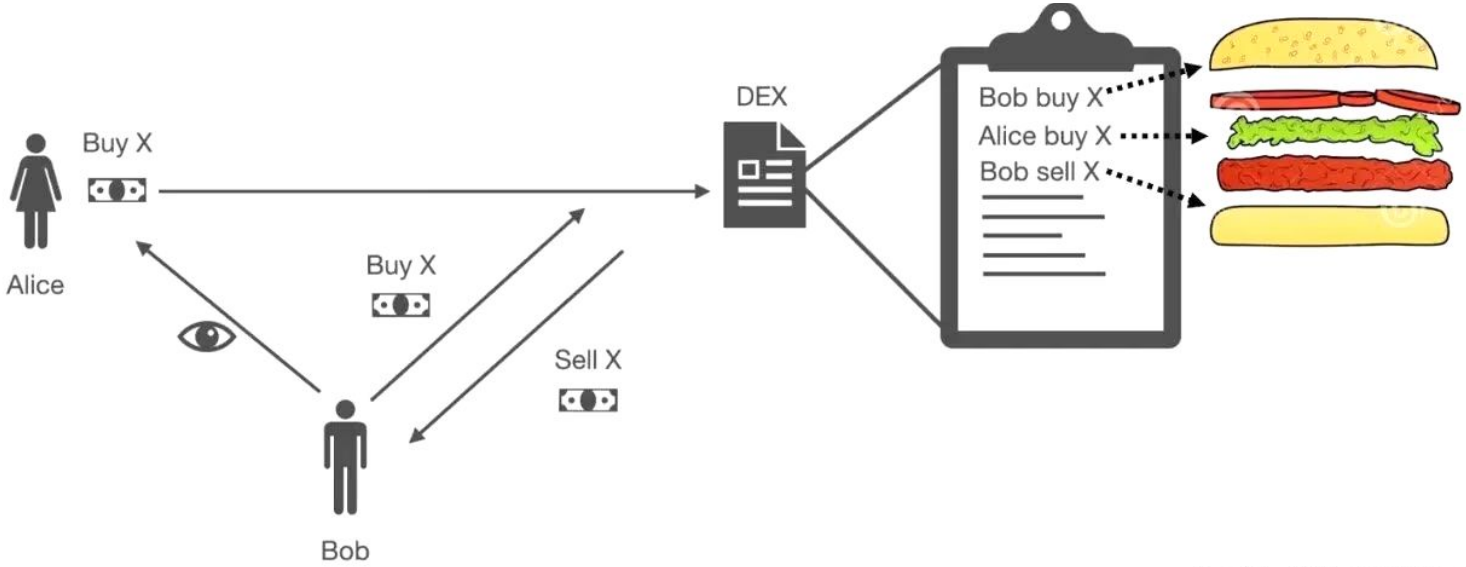
Offset	Variable	Data Type	Before	After
N/A	N/A	N/A	0x000000000000000000000000000000060564fd1f4f091ef5b94038ddfc8fba8	0x000000000000000000000000000000605650eaf1505cac6c034ab8265c684b

Privacy is Important.

Liquidation Hunting



MEV (Frontrunning Bots)

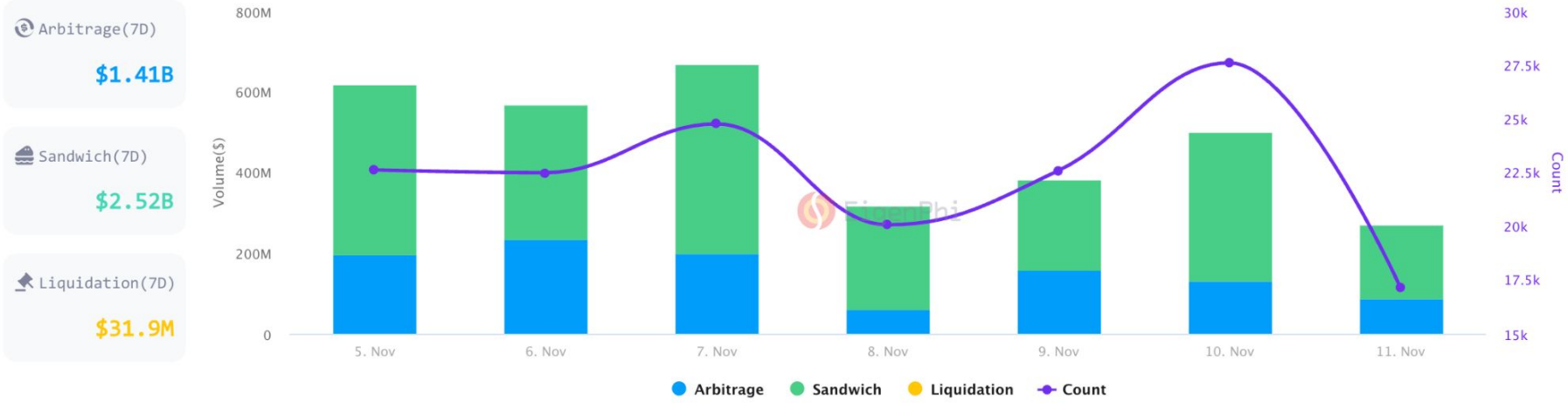


Source: Milk Road, Chorus One

MILKROAD

MEV (Frontrunning Bots)

Performance of MEV Types



Call for Action

CZ 🔶 **BNB** · Jun 2, 2025 ·

@cz_binance

Given recent events, I think now might be a good time for someone to launch a dark pool perp DEX.

I have always been puzzled with the fact that everyone can see your orders in real-time on a DEX. The problem is worse on a perp DEX where there are liquidations.

Even with a CEX...

Dark Pool

A privately organized financial forum or exchange for trading security.

- Helps with Price Discovery (Zhu, 2014)

On Chain but Private?

Encrypted Transactions

Address 0xd90e2f925da726b50c4ed8d0fb90ad053324f31b (Tornado.Cash: Router)

Name `EncryptedNote (index_topic_1 address sender, bytes encryptedNote)` View Source

Topics

0 `0xfa28df43db3553771f7209dcef046f3bdfea15870ab625dcda30ac58b82b4008`

1: sender Dec → `0x70f20D25Ca2B35a4D094b379cc08d73deE26cd47`

Data Dec Hex

e... FBBDBA0D09DEDBDDE447700C6FF6491225397F7026646193EF9AFC79B7821D1EACA37481CE8903A653D257A3CEEE07C7C1
1F123EB6D09367E30ED7C1DD8AEDBEDA47BD846E8EB61DD79E428D9DA47B1EBABFFE80D3593B12DDDAB540B7A9B7078735
EFB82444601AF9741B36A7B19CE12A0660A7A249847902D2F3EDB4614689CC42B8EDD8EC8B133E73F4CB33B32D18BD29A2
162F56232A086D81BC6B857F0B4948AE22A7B092BDD00B459A6BAF5E2E00665ECB8E58073800CC9C7188E6FAD89ABA0B79
D78B33F70EBECE434D1C3277082D20272457E1F8F9157972199CE4A13F0D09DE4FA7E71D633E93F2F6F0495925

Research Objectives

Implement **De**centralised **P**rivate **L**imit **O**rder **B**ook (DePLOB)

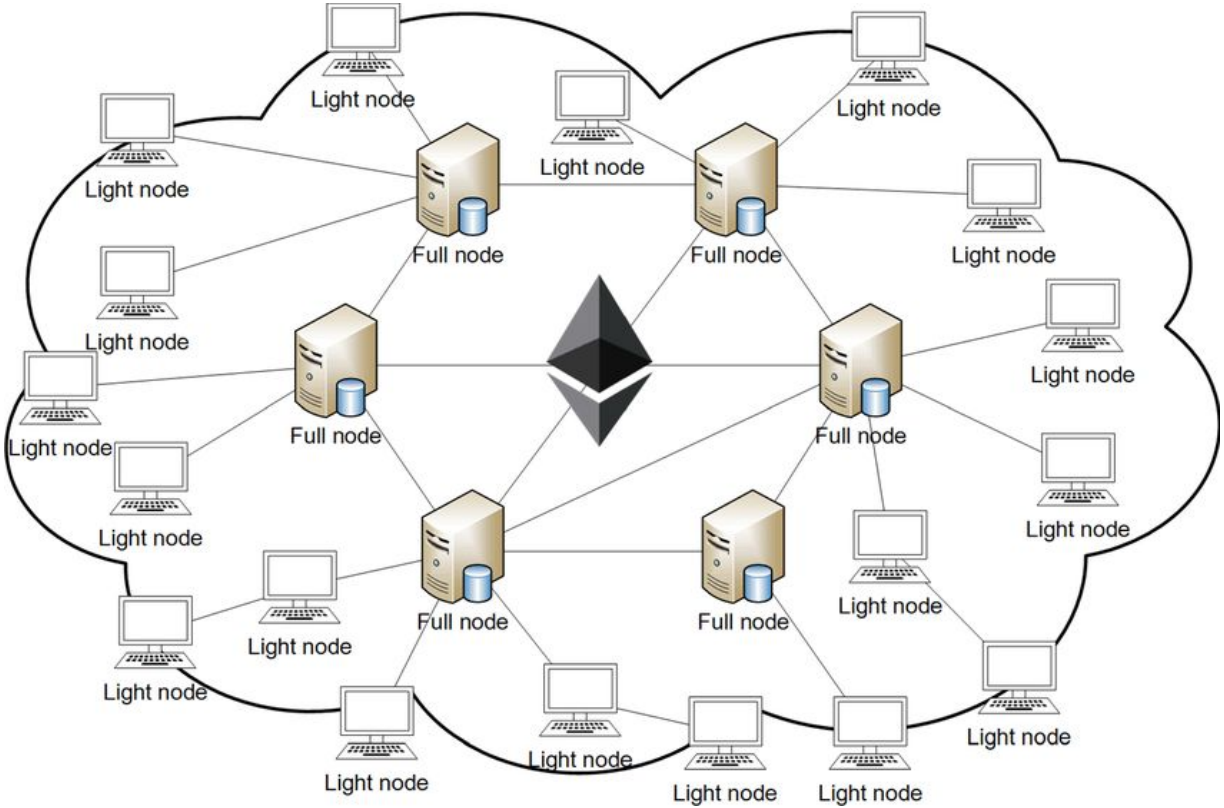
- Privacy Preserving Limit Orders
- On-Chain
- Verifiably Correct

Built With

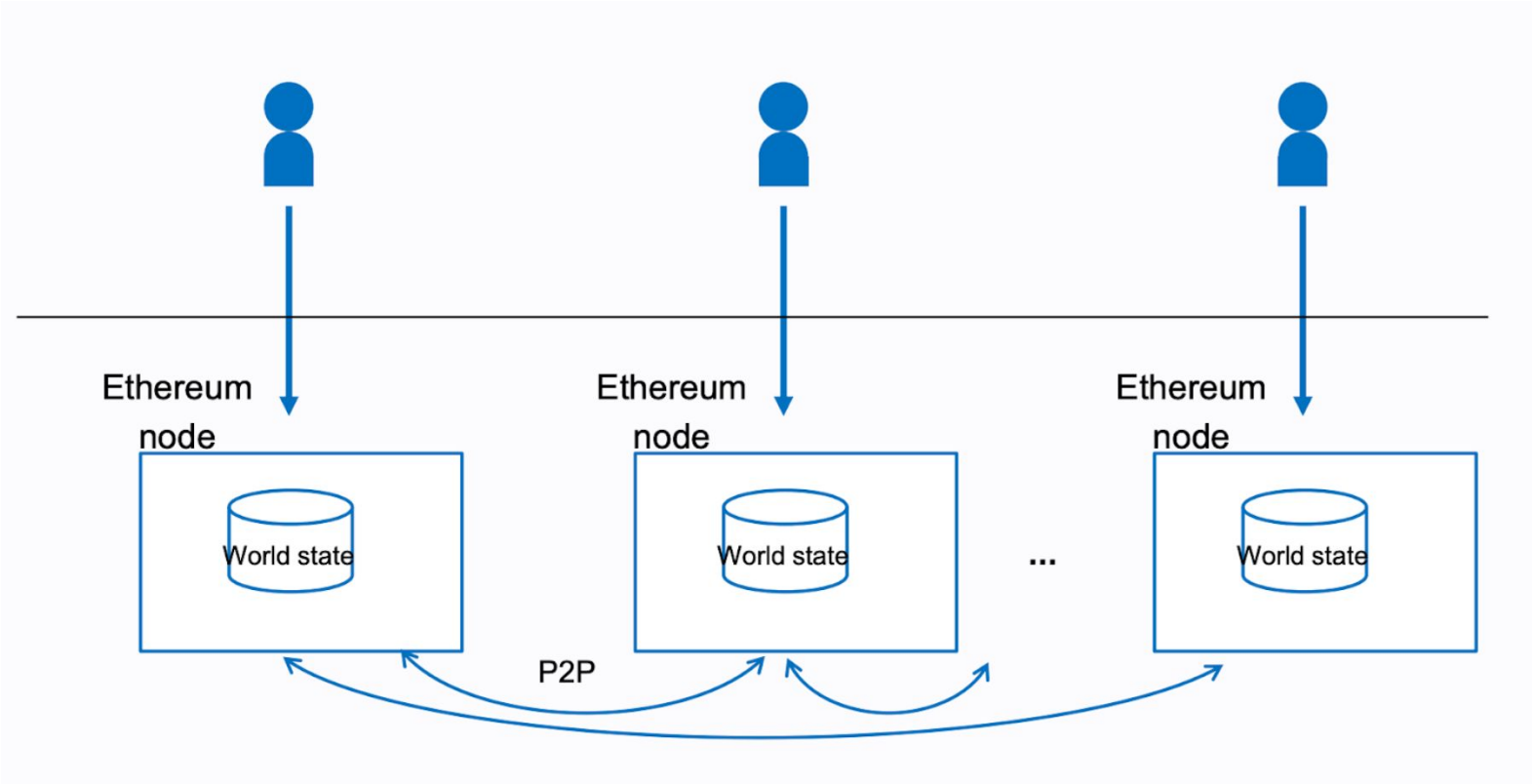
- Smart Contracts
- Trusted Execution Environment (TEE)
- Zero Knowledge Proof (ZKP)

Preliminary Knowledge

Blockchain: Consensus



Blockchain: Transactions



Blockchain: Smart Contracts

Transactions | Internal Transactions | Token Transfers (ERC-20) | NFT Transfers | Other Transactions | Contract | Events | Analytics | Assets | Cards New | Info | Advanced Filter

Code | Read Contract | Write Contract | Search Source Code

Similar Match Source Code

This contract matches the **deployed Bytecode** of the Source Code for Contract 0x8f8EF111...da062f163

The constructor portion of the code might be different and could alter the actual behaviour of the contract

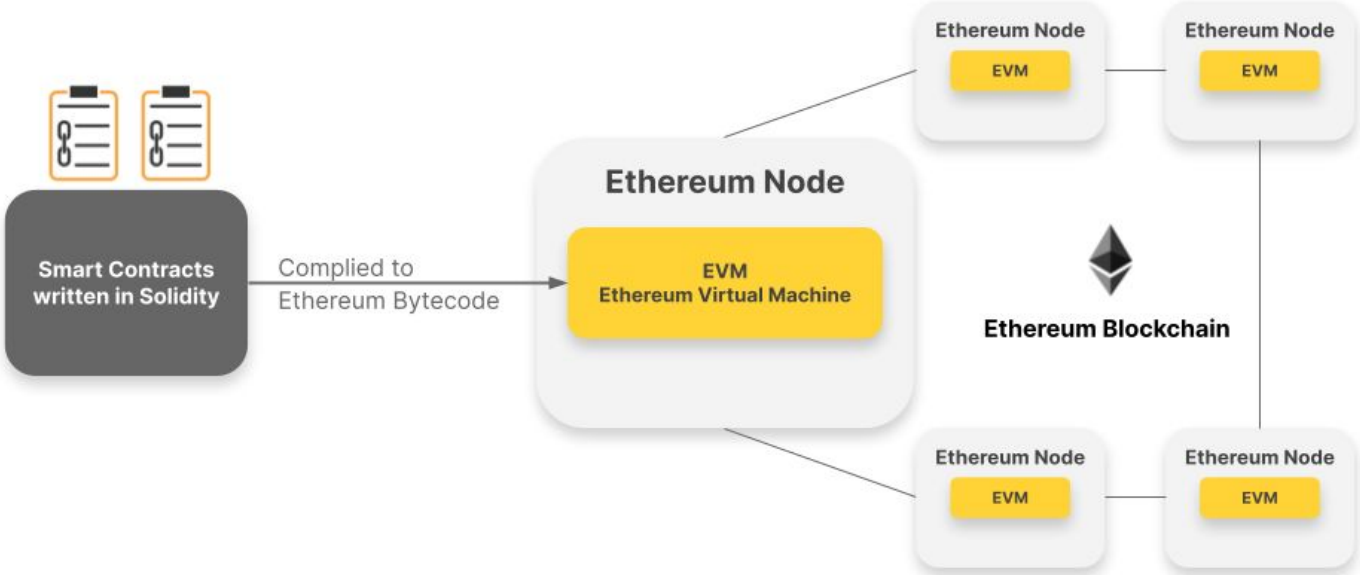
Contract Name:	**UniswapV3Pool**	Optimization Enabled:	**Yes** with **800** runs
Compiler Version	**v0.7.6+commit.7338295f**	Other Settings:	**default** evmVersion, **None** license

Contract Source Code (Solidity Standard Json-Input format) | Open In | More Options

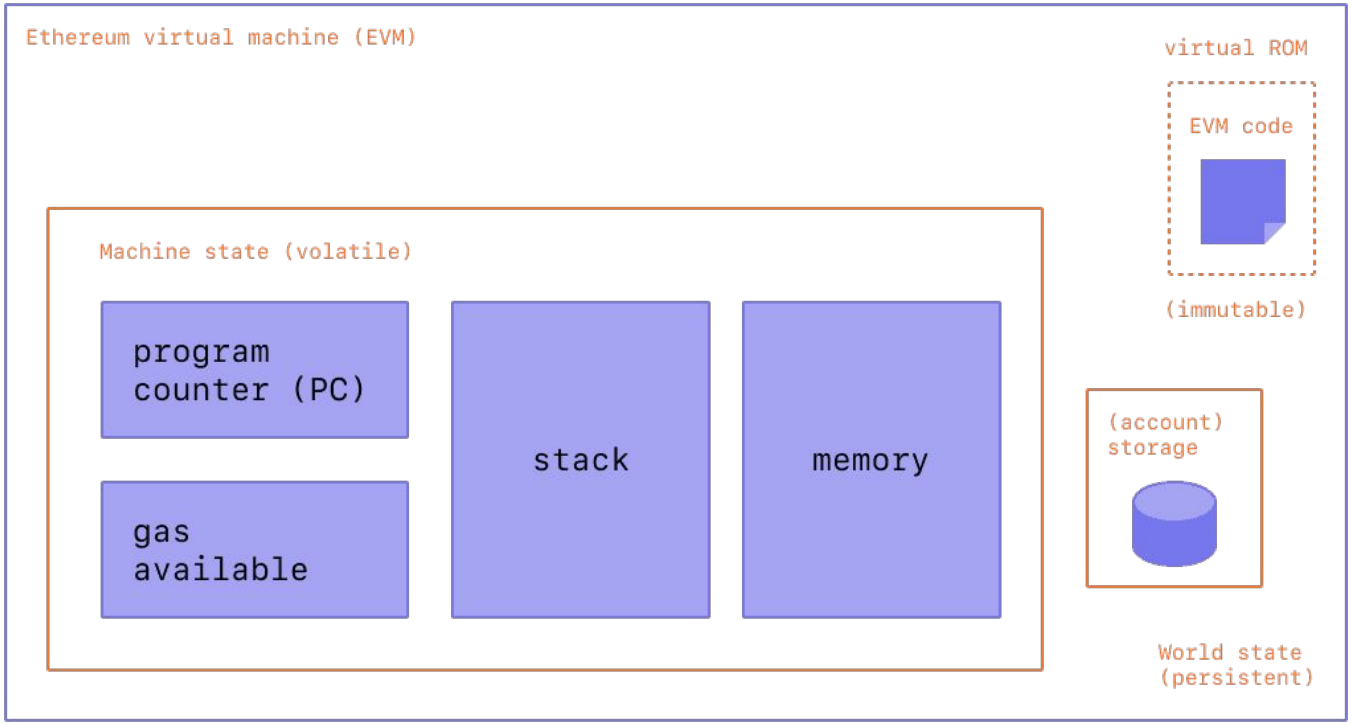
File 1 of 31 : UniswapV3Pool.sol | Outline

```
// SPDX-License-Identifier: BUSL-1.1
pragma solidity =0.7.6;

import './interfaces/IUniswapV3Pool.sol';

import './NoDelegateCall.sol';

import './libraries/LowGasSafeMath.sol';
import './libraries/SafeCast.sol';
import './libraries/Tick.sol';
import './libraries/TickBitmap.sol';
import './libraries/Position.sol';
import './libraries/Oracle.sol';

import './libraries/FullMath.sol';
import './libraries/FixedPoint128.sol';
import './libraries/TransferHelper.sol';
import './libraries/TickMath.sol';
import './libraries/LiquidityMath.sol';
import './libraries/SqrtPriceMath.sol';
import './libraries/SwapMath.sol';

import './interfaces/IUniswapV3PoolDeployer.sol';
import './interfaces/IUniswapV3Factory.sol';
import './interfaces/IERC20Minimal.sol';
```

Blockchain: EVM



Blockchain: EVM

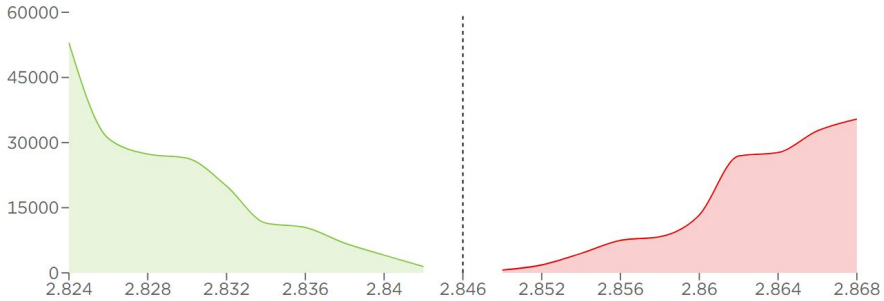


Central Limit Order Book (CLOB)

Order book

Order	Volume	Buy	Sell	Volume	Order
2	1 452	2,842	2,850	647	2
4	2 635	2,840	2,852	1 179	1
2	2 756	2,838	2,854	2 656	4
3	3 621	2,836	2,856	3 009	2
2	979	2,834	2,858	829	3
3	8 566	2,832	2,860	5 000	1
4	6 382	2,830	2,862	13 587	3
2	958	2,828	2,864	804	2
2	3 596	2,826	2,866	5 000	1
5	22 030	2,824	2,868	2 714	2

29 52 975 35 425 21



Central Limit Order Book (CLOB)

$$o_i = (p_i, q_i, s_i, t_i),$$

$$\mathscr{B} = \{o_i \mid s_i = \text{buy}\}, \quad \mathscr{A} = \{o_j \mid s_j = \text{sell}\},$$

stored in sorted lists according to *price-time priority*

Zero Knowledge Proofs (ZKP)

- A party (prover) convinces another party (verifier) that a certain statement is true without revealing any information aside from the truth of the information (Hasan, 2019)

$$\mathscr{P}(x, w) \Rightarrow \mathscr{V}(x) \quad \text{s.t.} \quad (x, w) \in \mathscr{R}.$$

Zero Knowledge Proofs: Completeness

$$\Pr[\mathscr{V} \text{ accepts } | (x, w) \in \mathscr{R}] = 1.$$

Zero Knowledge Proofs: Soundness

$$\Pr[\mathscr{V} \text{ accepts} \mid (x, w) \notin \mathscr{R}] \leq \varepsilon.$$

Zero Knowledge Proofs: Zero Knowledge

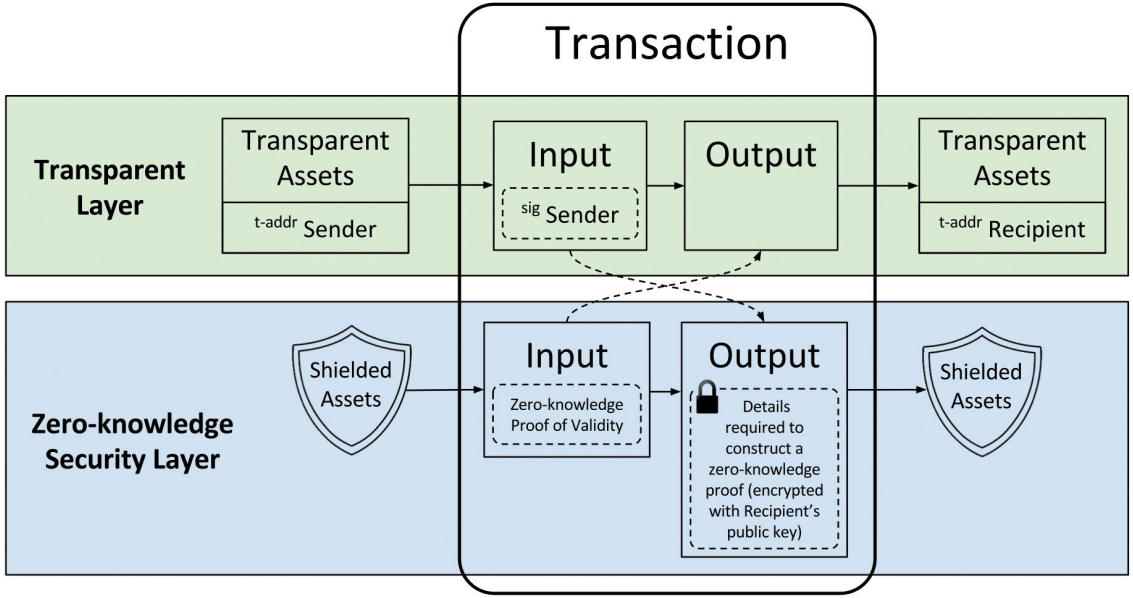
$$\mathrm{Sim}(\mathscr{S}(x)) \approx \mathrm{View}_{\mathscr{V}}(\mathscr{P}(x, w)).$$

Zero Knowledge Proofs: zk-SNARKS

$$\mathrm{Verify}(\mathsf{vk}, x, \pi) = \begin{cases} 1, & \text{if } \exists w : (x, w) \in \mathscr{R}, \\ 0, & \text{otherwise.} \end{cases}$$

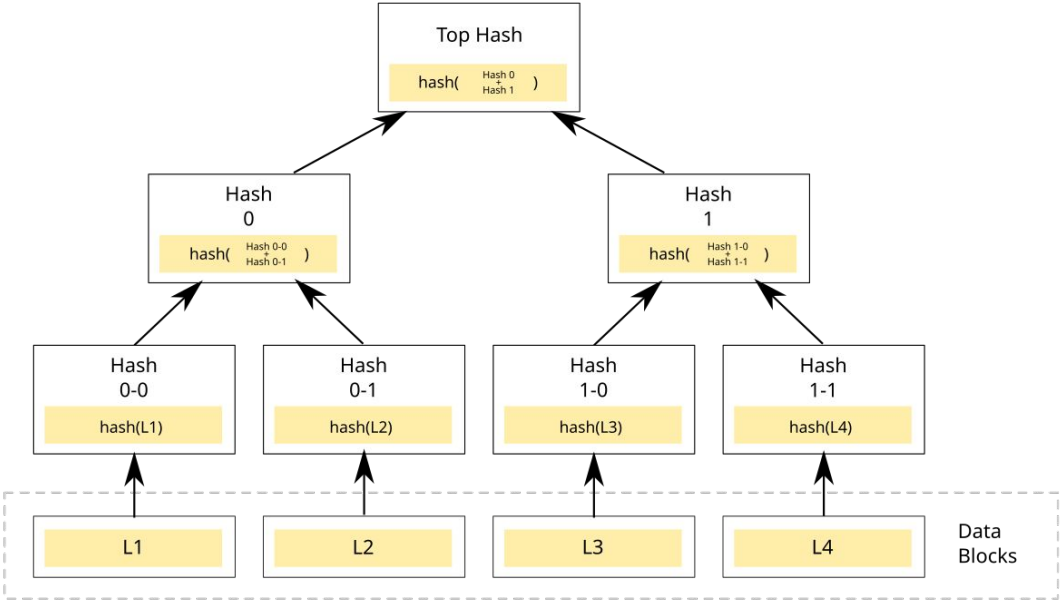
Zero Knowledge Proofs: Blockchain Application

- Guarantees **state changes** with anonymity (Ding, 2025)
- User can transfer tokens without disclosing the sender, recipient, or amount (ZCash)



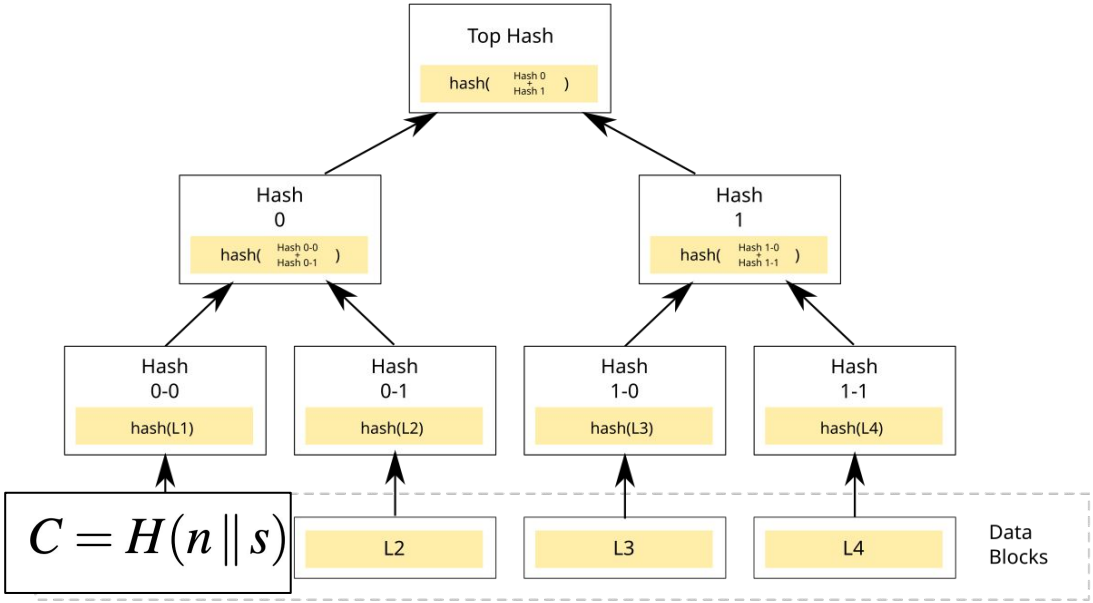
Merkle Tree

- A data structure used to efficiently verify the integrity and membership of large data sets with **Merkle Proofs** (root, leaf, path)



Merkle Tree: Nullifiers

- Used to ensure uniqueness and prevent **double spending** that depend on Merkle Trees.

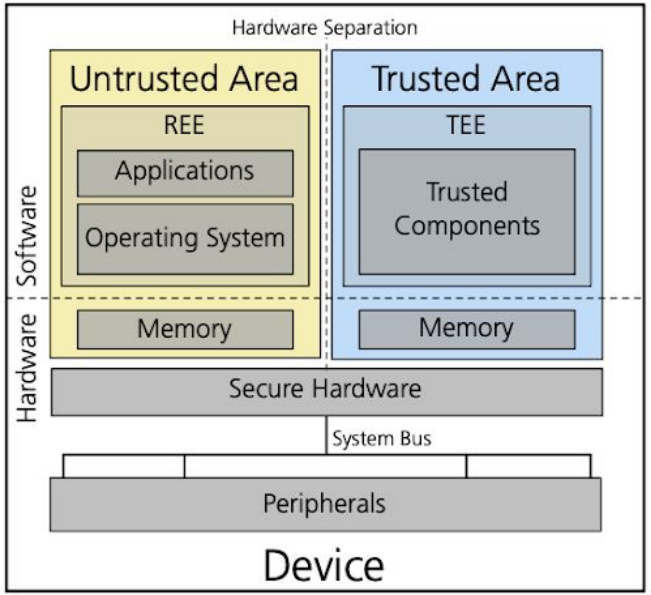


Merkle Tree: Nullifiers Proof

- Prove knowledge of (n, s) without revealing specific position of Commitment in Merkle Tree
  - `NullifierHash = H(Nullifier)`
  - $H(n \| s) \in MerkleTree$
- Simple Set Check to see if Nullifier hash has been previously used

Trusted Execution Environment (TEE)

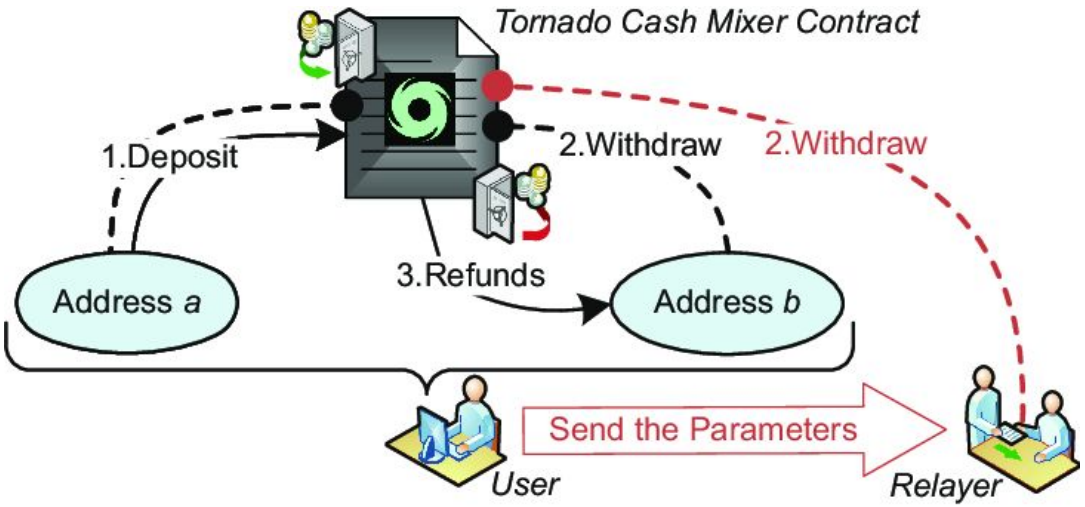
- **Hardware-enforced** secure area within a processor that guarantees the confidentiality and integrity of code and data loaded inside it (Sabt et al., 2015)



Case Studies

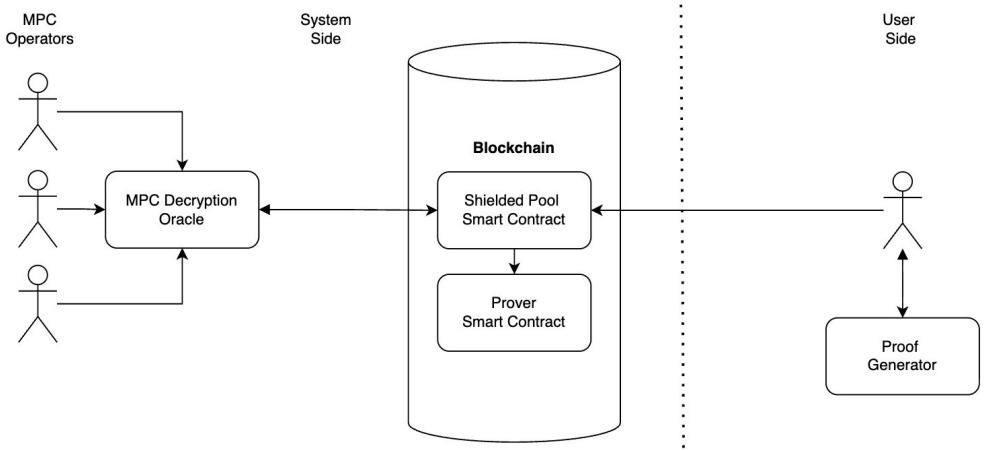
Tornado Cash

- Non custodial mixer where users can deposit and withdraw Ether with no reference to the deposit address (Pertsev et al., 2019)
- Pioneer of **Shielded Pools** with zk-SNARKS



COMMON: Order Book

- Privacy Preserving On-Chain Order Book (Garreta et al., 2023)
- Best ideation of the product
- Uses **Multi-Party Computation** which is a failing point



Recurring Limitations

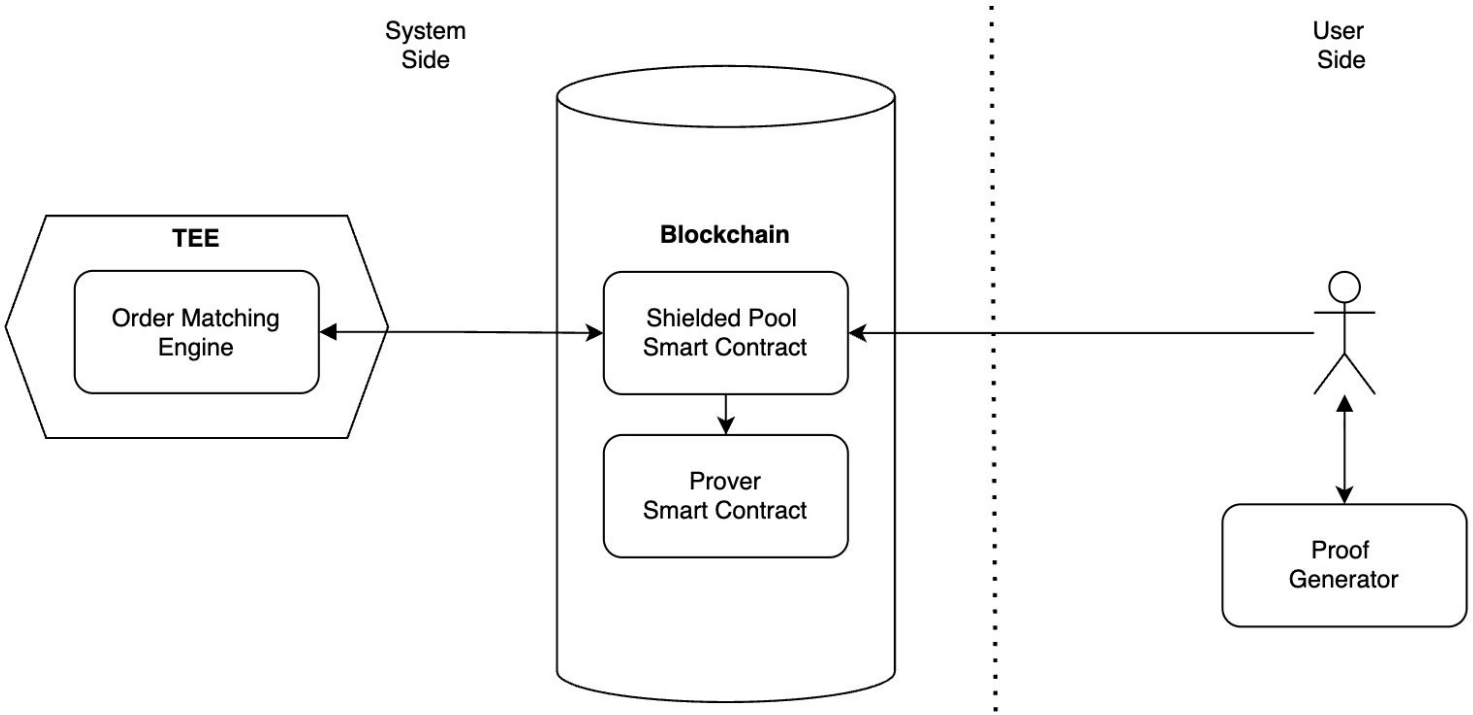
1. Trust in Decryption Entities (Attack Vector)
2. Batch-Based
3. Lack of Multi-Token and Routing Support

DePLOB

Features

- Privacy Preserving
- Multi Token Routing
- Verifiable
- Limit Order Book

System Diagram



Functions

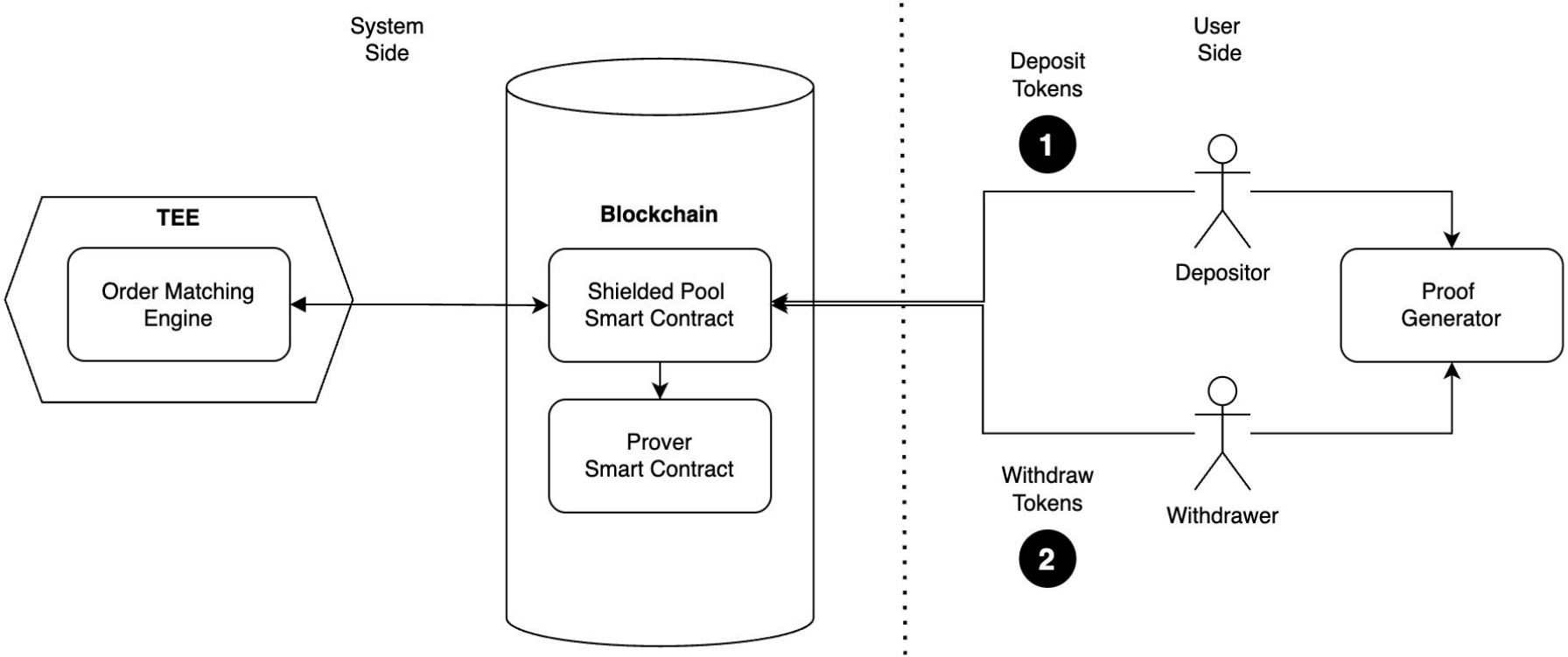
Smart Contract

- Deposit Tokens
- Withdraw Tokens
- Create Orders
- Cancel Orders
- Claim Orders

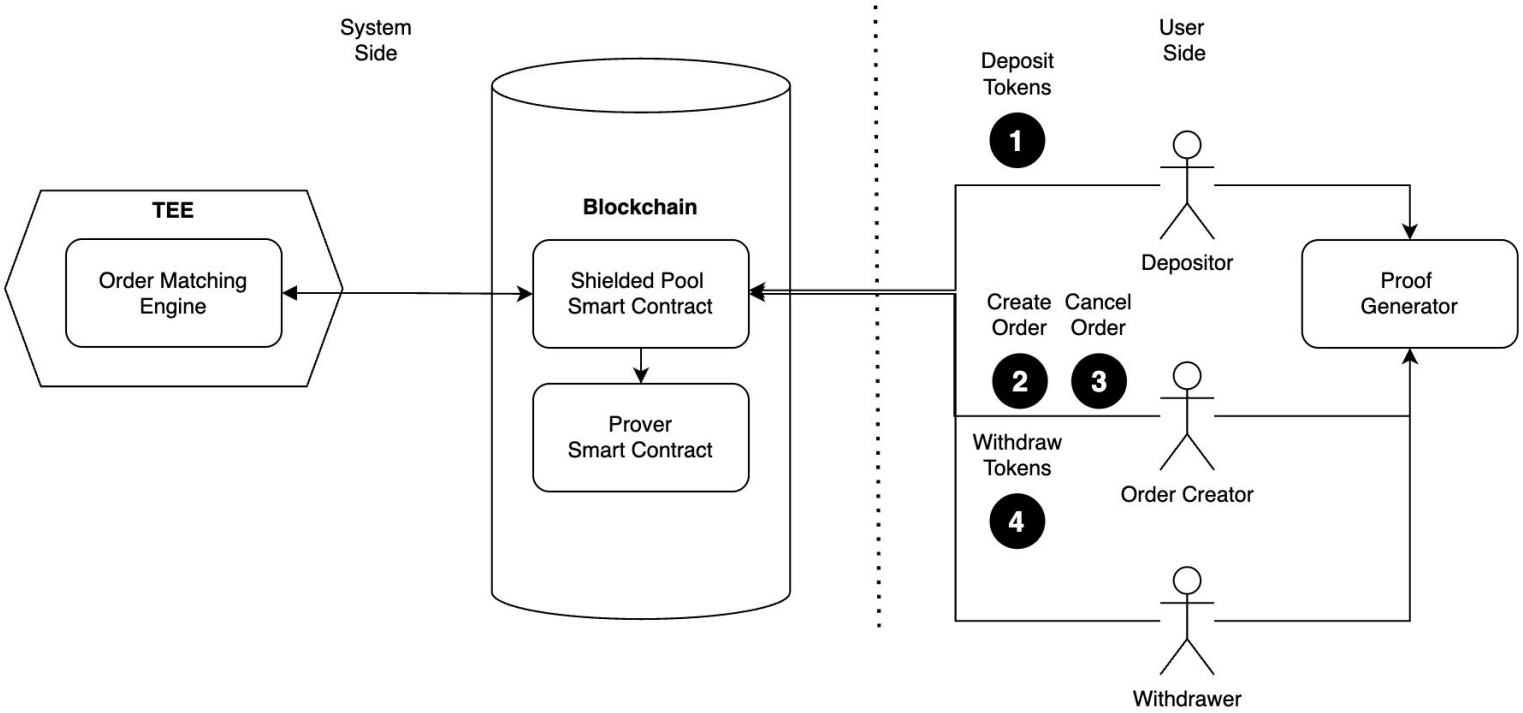
TEE

- Match Orders

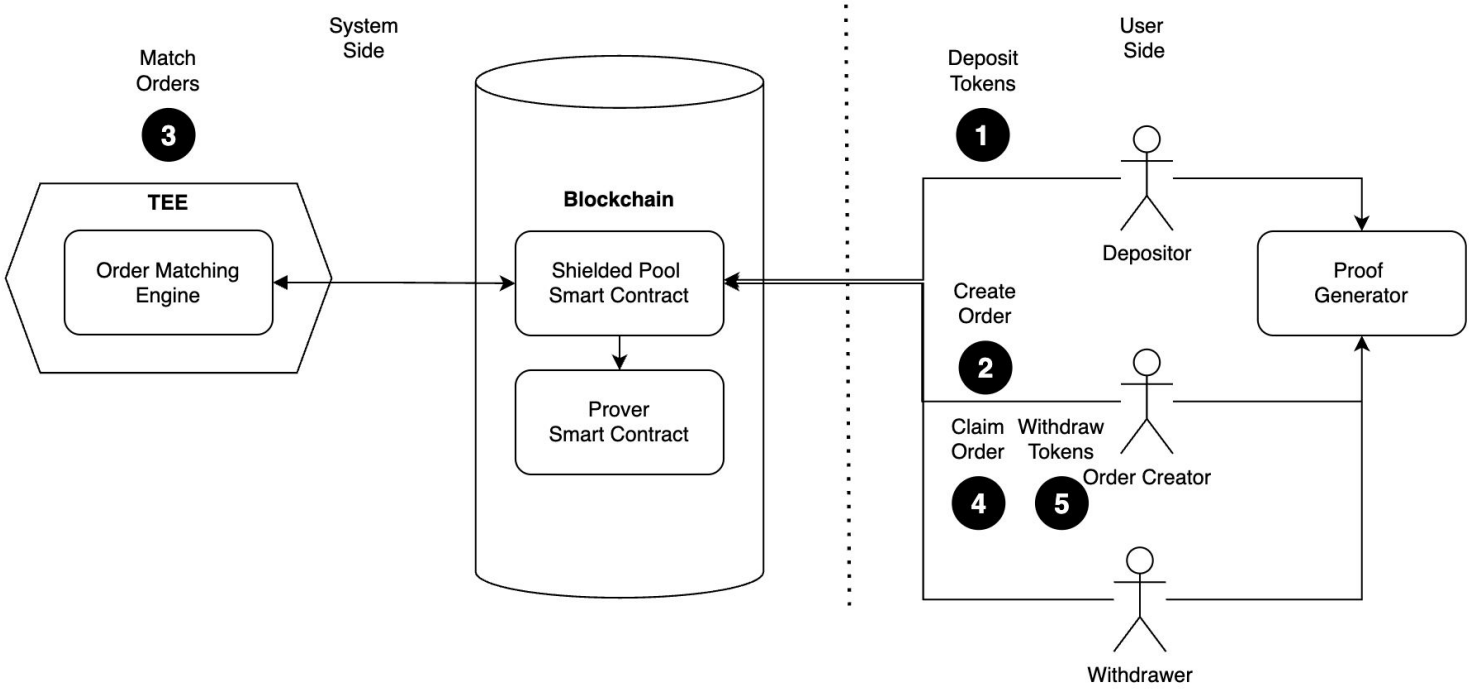
User Flow: Deposit & Withdraw



User Flow: Create Order, Cancel and Withdraw



User Flow: Create Order, Matched and Withdraw



Structures: Pair & Token

```
type Token = 0xString

struct Pair {
    from : Token
    to : Token
}
```

Structures: Order

```
struct Order {
    fillRatio : Ratio;
    pair : Pair;
    maxPrice : Price;
    isCancelled : Bool;
    isClaimed : Bool;
    encAmount : AHCipherText;
}
```

Structures: Note

```
struct Note {
    token : Token;
    amount : BigInt;
    secret : BigInt;
    nullifier : BigInt;

}
```

Structures: OrderNote

```
struct OrderNote {
    orderId : OrderId;
    pair : Pair;
    amount : BigInt;
    orderSecret : BigInt;
    orderNullifier : BigInt;

}
```

Deposit Tokens

Input: `amount, token, noteHash, proof`

1. Assert the caller has sent value of tokens along the transaction
2. Assert $0 \leq$ `amount` $\leq$ MAXSUPPLY
3. Set $x_{\texttt{DepositTokens}}$ = `(amount, token, noteHash)`
4. Assert `ZKP.V(`$R_{\texttt{DepositTokens}}$`,` $x_{\texttt{DepositTokens}}$`, proof)`
5. `self.tokenMerkleTree.addLeaf(noteHash)`

Withdraw Tokens

Input: `valueOut, token, root, proof, tokenNullifier, newNoteHash`

1. Assert `self.tokenMerkleTree.isHistoricalRoot(root)`
2. Assert `self.tokenNullifierSet.nullify(tokenNullifier)`
3. Set $x_{\texttt{WithdrawTokens}}$ = `(root, valueOut, token, newNoteHash, tokenNullifier)`
4. Assert `ZKP.V(`$R_{\texttt{WithdrawTokens}}$`, `$x_{\texttt{WithdrawTokens}}$`, proof)`
5. `self.tokenMerkleTree.addLeaf(newNoteHash)`
6. Send `valueOut` of `tokens` to the caller

Create Orders

Input: `orderId, pair, maxPrice, root, noteHash, orderHash, encAmount, proof, tokenNullifier`

1. Assert `orderId` $\notin$ `self.ordersSet`
2. Assert `self.tokenMerkleTree.isHistoricalRoot(root)`
3. Assign $x_{\texttt{CreateOrder}}$ = `(root, noteHash, tokenNullifier, orderHash, encAmount, self.encPKey)`
4. Assert `ZKP.V(`$R_{\texttt{CreateOrder}}$`, `$x_{\texttt{CreateOrder}}$`, proof)`
5. Assert `self.tokenNullifierSet.nullify(tokenNullifier)`
6. `self.orderMerkleTree.Add(orderHash)`
7. `self.ordersSet.add(order)`
   a. `order.fillRatio = 0,`
   b. `order.pair = pair,`
   c. `order.maxPrice = maxPrice,`
   d. `order.isCancelled = False,`
   e. `order.isClaimed = False,`
   f. `order.encAmount = encAmount`

Cancel Orders

Input: `orderId, root, proof`

1. Assert `self.orderMerkleTree.isHistoricalRoot(root)`
2. Assign $x_{\texttt{CancelOrder}}$ `= (root, orderId)`
3. Assert `ZKP.V(`$R_{\texttt{CancelOrder}}$`,` $x_{\texttt{CancelOrder}}$`, proof)`
4. Set `self.ordersSet[orderId].isCancelled` = `True`

Claim Orders

Input: `orderId, orderRoot, orderNullifier, noteHash, proof`

1. Assert `self.orderNullifierSet.nullify(orderNullifier)`
2. Set `order` = `self.ordersSet[orderId]`
3. Assert `self.orderMerkleTree.isHistoricalRoot(orderRoot)`
4. Assert `order.isClaimed` = `False`
5. Assign $x_{\texttt{ClaimOrder}}$ = `(orderId, orderRoot, orderNullifier, noteHash, order.fillRatio)`
6. Assert `ZKP.V(`$R_{\texttt{ClaimOrder}}$`, `$x_{\texttt{ClaimOrder}}$`, proof)`
7. Set `self.ordersSet[orderId].isClaimed` = `True`
8. `self.tokenMerkleTree.Add(noteHash)`

Order Matching

TEE Order Matching Engine (Event Driven)

1. Assign `newOrder` = `self.ExtractEvent(event)`
2. Set `self.orderSet.Add(newOrder)`
3. Set `self.orderBook` = `self.UpdateOrder(newOrder)`
4. Assign `matchedOrders` = `self.InternalMatch()`
5. Generate proof π from `matchedOrders` and `orderBook`
6. For `matched` in `matchedOrders`:
    - `OrderBook.UpdateOrder(matched, proof)`

Plan Next Semester

Objectives	Weeks													
	1	2	3	4	5	6	Recess Week	7	8	9	10	11	12	13
Zero Knowledge Proof Circuits	**Implement zk-SNARKS for all functions**								**Integration**		**Evaluation and Results**			
Smart Contract				**Implement smart contract for all required functions**										
TEE Matching Engine							**Implement TEE matching engine** 1. Price Time Priority 2. Multi Token Support							
Interface									**Frontend for users to use the functions**					
Report				**Report Writing**										

Q&A

Appendix

$R_{\text{DepositTokens}}$

Instance: amount, tokenAdr, noteHash

Witness: note

Constraints:

- note.tokenAdr = tokenAdr
- note.amount = amount
- noteHash = Hash(tokenId, amount, note.tokenSecret, note.tokenNullifier)

$R_{\text{WithdrawTokens}}$

Instance: root, valueOut, tokenAdr, newNoteHash, tokenNullifier

Witness: noteHash, path, note, newNote

Constraints:

- newNoteHash = Hash(newNote)
- (root, Hash(note), path) $\in$ $\text{MerkleProof}_{\text{TokenMerkleTree}}$
- note.tokenNullifier = tokenNullifier
- note.tokenAdr = tokenAdr = newNote.tokenAdr
- 0 ≤ newNote.value ≤ MAXSUPPLY
- newNote.value + valueOut = note.value

$R_{CreateOrder}$

Instance: root, noteHash, tokenNullifier, orderHash, encAmount, encPKey

Witness: note, orderNote, path, r

Constraints:

- orderHash = Hash (orderNote)
- noteHash = Hash(note)
- (root, noteHash, path) $\in$ $MerkleProof_{TokenMerkleTree}$
- note.{tokenAddress, amount} = orderNote.{tokenAddress, amount}
- note.tokenNullifier = tokenNullifier
- encAmount = TEE.Encrypt(encPKey, orderNote.amount, r)

$R_{CancelOrder}$

Instance: root, orderId

Witness: orderNote, orderNoteHash, path,

Constraints:

- (root, orderNoteHash, path) $\in$ $MerkleProof_{OrderMerkleTree}$
- orderNote.orderId = orderId

$R_{ClaimOrder}$

Instance:orderId, root, orderNullifier, noteHash, fillRatio

Witness: orderNote, path, note

Constraints:

- (root, orderNoteHash, path) $\in$ $MerkleProof_{OrderMerkleTree}$
- orderNote.orderId = orderId
- note.TokenAdr = orderNote.pair.from
- (1-fillRatio) x orderNote.amount = note.amount
- noteHash = Hash(note)
- orderNullifier = hash(orderNote.Nullifier)

Outline

1. Problem Motivation
    a. Rise of Blockchain
    b. On Chain Swaps with smart contract
    c. Importance of Privacy, liquidation example
    d. Dark Pools
2. Research
    a. Blockchain
    b. DeFi
    c. CLOB
    d. ZKP
        i. 3 Attributes
        ii. Where's Waldo example
    e. Merkle Trees
    f. Nullifiers
    g. TEE
3. Case Studies
    a. Tornado Cash
    b. COMMON
    c. Common Limitations
4. DePLOB
    a. Problem Formulation
    b. Progress made so far (system diagram)
5. Plan Next Semester